

# KQL (Kusto Query Language) Cheat Sheet

58 core KQL keywords, operators, and functions with descriptions and examples.

| Keyword              | Category            | Description                                               | Example                                               |
|----------------------|---------------------|-----------------------------------------------------------|-------------------------------------------------------|
| <b>where</b>         | <i>Filtering</i>    | Filters rows based on a predicate.                        | Table   where EventID == 4688                         |
| <b>==</b>            | <i>Comparison</i>   | Equals (case-sensitive).                                  | where Image == "cmd.exe"                              |
| <b>!=</b>            | <i>Comparison</i>   | Not equal.                                                | where EventID != 4688                                 |
| <b>==~</b>           | <i>Comparison</i>   | Equals, case-insensitive.                                 | where Image ==~ "CMD.EXE"                             |
| <b>!~</b>            | <i>Comparison</i>   | Not equal, case-insensitive.                              | where Image !~ "cmd.exe"                              |
| <b>contains</b>      | <i>String match</i> | Contains substring, case-insensitive.                     | where CommandLine contains "powershell"               |
| <b>!contains</b>     | <i>String match</i> | Does not contain substring.                               | where CommandLine !contains "test"                    |
| <b>contains_cs</b>   | <i>String match</i> | Contains substring, case-sensitive.                       | where Image contains_cs "System32"                    |
| <b>has</b>           | <i>String match</i> | Contains as a whole term (indexed, faster than contains). | where CommandLine has "powershell"                    |
| <b>!has</b>          | <i>String match</i> | Does not contain as a whole term.                         | where CommandLine !has "cmd"                          |
| <b>hasprefix</b>     | <i>String match</i> | Has a term starting with the given string.                | where Image hasprefix "power"                         |
| <b>hassuffix</b>     | <i>String match</i> | Has a term ending with the given string.                  | where Image hassuffix ".exe"                          |
| <b>startswith</b>    | <i>String match</i> | Field starts with substring.                              | where Image startswith "C:\Windows"                   |
| <b>!startswith</b>   | <i>String match</i> | Field does not start with substring.                      | where Image !startswith "C:\Temp"                     |
| <b>endswith</b>      | <i>String match</i> | Field ends with substring.                                | where Image endswith ".exe"                           |
| <b>!endswith</b>     | <i>String match</i> | Field does not end with substring.                        | where Image !endswith ".dll"                          |
| <b>matches regex</b> | <i>String match</i> | Matches a regular expression.                             | where CommandLine matches regex @"https?:/"           |
| <b>in</b>            | <i>Membership</i>   | Value exists in a given list.                             | where EventID in (4624, 4625, 4688)                   |
| <b>!in</b>           | <i>Membership</i>   | Value does not exist in a given list.                     | where EventID !in (4634, 4647)                        |
| <b>between</b>       | <i>Range</i>        | Value falls within a range.                               | where TimeGenerated between (ago(1d) .. now())        |
| <b>and / or</b>      | <i>Logical</i>      | Combine multiple predicates.                              | where EventID == 4688 and Image contains "powershell" |

| Keyword               | Category                    | Description                                                 | Example                                                    |
|-----------------------|-----------------------------|-------------------------------------------------------------|------------------------------------------------------------|
| <b>sort by</b>        | <i>Sorting</i>              | Sorts results by one or more columns.                       | Table   sort by TimeGenerated desc                         |
| <b>order by</b>       | <i>Sorting</i>              | Alias for sort by.                                          | Table   order by Count desc                                |
| <b>top</b>            | <i>Sorting + limiting</i>   | Returns the top N rows by a column.                         | Table   top 10 by Count desc                               |
| <b>take</b>           | <i>Row limiting</i>         | Returns an arbitrary N rows (no guaranteed order).          | Table   take 20                                            |
| <b>limit</b>          | <i>Row limiting</i>         | Alias for take.                                             | Table   limit 50                                           |
| <b>distinct</b>       | <i>Deduplication</i>        | Returns unique combinations of the given columns.           | Table   distinct Image, User                               |
| <b>project</b>        | <i>Column selection</i>     | Selects and optionally renames columns.                     | Table   project TimeGenerated, Computer, Image             |
| <b>project-away</b>   | <i>Column selection</i>     | Removes the specified columns from output.                  | Table   project-away ProcessGuid                           |
| <b>project-rename</b> | <i>Column selection</i>     | Renames columns while keeping the rest.                     | Table   project-rename Host = Computer                     |
| <b>extend</b>         | <i>Column creation</i>      | Adds a new computed column.                                 | Table   extend Hour = datetime_part("hour", TimeGenerated) |
| <b>summarize</b>      | <i>Aggregation</i>          | Groups rows and applies aggregation functions.              | Table   summarize count() by Image                         |
| <b>count()</b>        | <i>Aggregation function</i> | Counts rows in each group.                                  | Table   summarize count() by EventID                       |
| <b>count</b>          | <i>Standalone operator</i>  | Returns the total row count of the result.                  | Table   count                                              |
| <b>dcount()</b>       | <i>Aggregation function</i> | Counts distinct values.                                     | Table   summarize dcount(Computer)                         |
| <b>sum()</b>          | <i>Aggregation function</i> | Sums a numeric column per group.                            | Table   summarize sum(BytesSent) by Computer               |
| <b>avg()</b>          | <i>Aggregation function</i> | Averages a numeric column per group.                        | Table   summarize avg(Duration) by User                    |
| <b>min() / max()</b>  | <i>Aggregation function</i> | Smallest / largest value per group.                         | Table   summarize max(TimeGenerated) by Computer           |
| <b>bin()</b>          | <i>Bucketing</i>            | Rounds values into fixed-size buckets, often used for time. | Table   summarize count() by bin(TimeGenerated, 1h)        |
| <b>join</b>           | <i>Combining tables</i>     | Combines rows from two tables on a matching key.            | T1   join kind=inner (T2) on ProcessGuid                   |
| <b>union</b>          | <i>Combining tables</i>     | Stacks rows from multiple tables together.                  | SecurityEvent   union Sysmon                               |
| <b>let</b>            | <i>Variables</i>            | Defines a reusable variable or subquery.                    | let threshold = 5; Table   where Count > threshold         |

| Keyword                          | Category                 | Description                                                | Example                                                                  |
|----------------------------------|--------------------------|------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>ago()</b>                     | <i>Date/time</i>         | A time offset back from now.                               | where TimeGenerated > ago(7d)                                            |
| <b>now()</b>                     | <i>Date/time</i>         | The current UTC time.                                      | extend CheckedAt = now()                                                 |
| <b>datetime_part()</b>           | <i>Date/time</i>         | Extracts a part (hour, day, etc.) from a datetime.         | extend Hr = datetime_part("hour", TimeGenerated)                         |
| <b>format_datetime()</b>         | <i>Date/time</i>         | Formats a datetime value as a string.                      | extend Formatted = format_datetime(TimeGenerated, "yyyy-MM-dd")          |
| <b>case()</b>                    | <i>Conditional logic</i> | Multi-branch conditional expression.                       | extend Severity =<br>case(Level==1, "Critical", Level==2, "High", "Low") |
| <b>iff()</b>                     | <i>Conditional logic</i> | Inline if/else for a single condition.                     | extend IsSystem = iff(User == "SYSTEM", "Yes", "No")                     |
| <b>strcat()</b>                  | <i>String function</i>   | Concatenates strings together.                             | extend FullPath = strcat(Directory, "\\",<br>FileName)                   |
| <b>split()</b>                   | <i>String function</i>   | Splits a string into an array on a delimiter.              | extend Parts = split(CommandLine, " ")                                   |
| <b>substring()</b>               | <i>String function</i>   | Extracts part of a string.                                 | extend Ext = substring(FileName,<br>indexof(FileName, "."))              |
| <b>tolower() /<br/>toupper()</b> | <i>String function</i>   | Converts string case.                                      | extend ImageLower = tolower(Image)                                       |
| <b>tostring() /<br/>toint()</b>  | <i>Type conversion</i>   | Converts a value to string or integer.                     | extend PortNum = toint(SourcePort)                                       |
| <b>parse</b>                     | <i>Extraction</i>        | Extracts structured fields out of unstructured text.       | parse CommandLine with * "-Uri " Url " " *                               |
| <b>mv-expand</b>                 | <i>Array expansion</i>   | Expands a dynamic/array column into multiple rows.         | Table   mv-expand ParsedArray                                            |
| <b>render</b>                    | <i>Visualization</i>     | Renders results as a chart (in supported UIs).             | Table   summarize count() by<br>bin(TimeGenerated,1h)   render timechart |
| <b>datatable</b>                 | <i>Inline data</i>       | Creates a small static table inline for testing/joins.     | datatable(EventID:int, Name:string) [4688,<br>"ProcessCreate"]           |
| <b>print</b>                     | <i>Output</i>            | Outputs a single evaluated expression, useful for testing. | print ago(1d)                                                            |

Note: KQL is used in Microsoft Sentinel, Azure Data Explorer, and Log Analytics. Examples use generic field names (Image, CommandLine, TimeGenerated, Computer) that map to Sentinel/Sysmon-style tables — adjust field names to match your actual schema.